An Intelligent Waste Collection System Leveraging IoT and Real-Time Monitoring

Nikita Patil
Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning)
AP Shah Institute of Technology
Thane, India
nikitapatil79@apsit.edu.in

Neha Sirsat
Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning)
AP Shah Institute of Technology
Thane, India
nehasirsat80@apsit.edu.in

Aryan Kharat
Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning)
AP Shah Institute of Technology
Thane, India
23206001@apsit.edu.in

Prof. Sumeet Shingi
Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning)
AP Shah Institute of Technology
Thane, India
svshingi@apsit.edu.in

Abstract— Managing municipal waste is a tough task using old methods like fixed collection routes. These methods often lead to overflowing bins and wasted fuel. In this project, we built an "IoT-Based Smart Waste Management System" to solve these issues. We used ESP32 microcontrollers connected to Ultrasonic sensors (to check garbage levels) and Flame sensors (to detect fire).

For the software, we created a full-stack web application using Node.js and Express.js. We stored all the data in a MongoDB Atlas cloud database. A key feature of our system is security; we used JSON Web Tokens (JWT) to make sure only authorized people can access it. The system provides three different dashboards: one for officers to monitor bins, one for drivers to see routes, and one for citizens to report problems. By testing this prototype, we showed that automating waste collection can save money and keep the city cleaner.

Keywords— IoT, Waste Management, Smart Bin, Realtime Monitoring, Role-based Access.

I. Introduction

Managing solid waste has become a serious challenge for modern society. This is mainly due to rapid urbanization, industrial growth, and the fast-growing population in both cities and rural areas. The old ways of managing waste rely on checking bins manually and collecting garbage on fixed schedules. These methods are becoming inefficient, often resulting in overflowing bins, environmental pollution, and serious risks to public health.

Currently, municipal authorities use traditional methods where garbage trucks follow strict routes every day, regardless of whether the bins are full or empty. This approach wastes a lot of time, labor, and fuel. It also increases operational costs and carbon emissions. To solve the need for a better and sustainable solution, this project introduces an IoT-Based Smart Waste Management System. This system changes the traditional infrastructure into an intelligent, automated network.

By using the Internet of Things (IoT), we created a "smart city" environment where devices can exchange data without needing human help. The technical design of our system is divided into three main layers: 1. Device Layer: We used ESP32 microcontrollers connected to ultrasonic sensors to monitor how full the bins are, and flame sensors to detect fire hazards. 2. Application Layer: This is powered by a Node.js and Express.js backend, along with a responsive frontend interface. 3. Cloud Layer: We utilized MongoDB Atlas for scalable data storage and Vercel to handle continuous updates and deployment.

Our system uses Role-Based Access Control (RBAC) to provide specific dashboards for different users. Municipal Officers can plan schedules based on real-time data, Drivers can follow better routes, and Citizens can report local issues. To keep the system secure, we implemented JSON Web Tokens (JWT) for encrypted authentication, which stops unauthorized access to sensitive data.

By using RESTful APIs to connect the hardware with the software, our system removes the need for manual inspection. This directly reduces fuel consumption and makes the process more transparent. Ultimately, this project is a strong example of how digital technology can solve real-world urban problems, helping to build smarter, cleaner, and more efficient cities for the future.

II. Literature Survey

Ideally, smart cities need smart waste management. Many researchers have tried to solve the garbage collection problem using IoT. For example, Navghane et al. [1] and Kumar & Jain [8] showed how ultrasonic sensors can measure garbage levels. Their work proved that using sensors is better than manual checking. However, early projects like these only focused on the hardware and sensors.

Later, Longhi et al. [2] suggested using wireless networks to connect these bins. But as Anagnostopoulos et al. [3] pointed out, sending data from thousands of bins can be difficult without a good server.

Modern solutions try to fix this by using cloud databases. Chodorow [13] mentioned that databases like MongoDB are great for handling large amounts of sensor data. Also,

simply measuring the garbage level is not enough. We also need to check for safety issues like fire, which was discussed by Zanella et al. [6]. Recent advancements in municipal solid waste management have increasingly integrated Internet of Things (IoT) frameworks and machine learning algorithms. Borojeni and Abdi [16] designed an IoT-WMS system that utilizes ultrasonic sensors to determine bin capacity, seamlessly streamlining the routing process for garbage collection vehicles. To address the challenges of long-range communication and power efficiency, Laha et al. [17] proposed a framework combining IoT with LoRaWAN technology to optimize collection routes in green city projects. Furthermore, Gurupriya and Yadav [18] demonstrated the cost-effectiveness of using ESP8266 microcontrollers for real-time waste monitoring and cloud synchronization. Moving beyond basic collection, Uthayakumar et al. [19] integrated AI and IoT sensors to achieve automated waste segregation, highlighting the potential for advanced recycling mechanisms. Finally, Alowayr [20] emphasized that deploying IoT-enabled smart bins alongside dynamic route selection algorithms significantly reduces operational fuel costs and minimizes environmental emissions in smart cities.

One big missing piece in many existing systems is security. Most projects do not strictly control who can access the data. Our project improves on these previous ideas by adding a secure login system (using JWT) and separating the dashboards for drivers and officials.

III. Limitations of Existing System

Currently, the municipal waste management system works on a manual and static model. Garbage collection trucks follow a fixed, pre-decided route every day, regardless of whether the dustbins are full or empty. This creates a major efficiency problem. Trucks often visit empty bins, which leads to a huge waste of fuel, labor, and valuable time. Since there is no real-time visibility, authorities do not know when a bin is full. As a result, bins often overflow in busy areas, causing environmental pollution, bad odors, and serious health risks for the public. Additionally, the traditional system has no safety features; for example, if a fire breaks out inside a bin due to waste heat, there is no way to detect it immediately. These unoptimized operations increase the operational cost for the city and also increase the carbon footprint, making the current method unsustainable for modern cities.

Although our proposed "IoT-Based Smart Waste Management System" solves these issues using automation, it also has some inherent limitations. First, the system relies heavily on a stable internet connection. If the network is weak, the ESP32 microcontroller cannot send data to the MongoDB cloud database. Second, the hardware components, specifically the ultrasonic and flame sensors, are sensitive to the environment. Dust, moisture, or heavy rain can affect their accuracy over time, so they require regular maintenance. Furthermore, while our web application allows officers to monitor bins, the current version does not support automatic waste segregation (separating wet and dry waste). Also, setting up this IoT network across an entire city requires a significant initial investment. Lastly, the system currently suggests routes based on data, but it requires future integration with live GPS navigation to fully automate the driving directions for truck drivers.

IV. Problem Statement and Objectives

A. Problem Statement:

The amount of waste generated in cities is increasing rapidly, and the traditional methods used by municipal corporations are struggling to keep up. Currently, garbage collection relies on static schedules where trucks visit areas blindly. Since there is no way to check the bin status remotely, trucks often arrive too late, leading to overflowing bins that create unsanitary conditions and bad odors. On the other hand, trucks sometimes visit empty bins, which is a waste of fuel and manpower. To solve these inefficiency problems, this project presents an IoT-based Waste Management Solution. We use ultrasonic sensors to check fill levels and flame sensors to detect fire risks. By connecting a Node.js backend with a MongoDB database, our system allows for data-based route planning. This smart approach minimizes human error, saves fuel, and improves public health by ensuring timely cleaning.

B. Objectives

- To build a working smart waste management system that uses IoT sensors to track the status of garbage bins in real-time.
- To accurately measure how full a bin is using ultrasonic sensors and detect any fire accidents using flame sensors for safety.
- To create an easy-to-use web application that provides specific dashboards for municipal officers, drivers, and citizens to manage and view data.

V. Proposed System

A. System Overview

Our "IoT-Based Smart Waste Management System" is built using three main components: the hardware sensors, a web application with multiple roles (Officer, Driver, Citizen), and a cloud database for analytics. In the hardware section, we used the ESP32 microcontroller connected to ultrasonic and flame sensors. These sensors constantly check the bin levels and fire status and send this data to the cloud. Municipal officers have a main dashboard where they can see live updates and plan schedules. Drivers get a separate view to see their routes. Citizens also have a portal to check bin locations. The entire backend runs on Node.js and Express.js with MongoDB Atlas storing the data. This setup ensures that data flows smoothly from the bin to the user, using secure login methods (JWT) to keep the information safe.

B. Analytics Features

The admin dashboard is designed to show the following reports:

- Perception Layer (Hardware): This includes the ESP32 microcontroller with Ultrasonic sensors (for checking trash volume) and Flame sensors (for fire safety). These act as the eyes of the system, capturing live data from the environment.

- Network & Transport Layer: We used the Wi-Fi feature of the ESP32 to send data to the server via API calls. This layer ensures that the connection between the bin and our Node.js server is fast and secure.
- Application & Analytics Layer: This is the website that users see, hosted on Vercel. It uses the MongoDB database to save records. It takes the raw numbers from sensors and turns them into easy-to-read graphs for the Municipal officers and Drivers.

C.COMPREHENSIVE EXPERIMENTAL SETUP AND SYSTEM INTEGRATION

The experimental setup for this architecture was designed as a multi-layered framework connecting physical sensor nodes with cloud-based infrastructure, creating a comprehensive ecosystem for municipal sanitation. A high-performance ESP32 microcontroller was utilized as the core hardware gateway due to its dual-core processing capabilities and integrated Wi-Fi module, making it highly suitable for IoT applications. For volumetric waste measurement, an HC-SR04 ultrasonic sensor was mounted at the top center of the bin lid. This sensor calculates the fill level by measuring the time-of-flight of ultrasonic pulses reflected from the waste surface. Additionally, a highly sensitive flame sensor was integrated to detect infrared emissions from potential fires, enabling the system to trigger immediate emergency alerts.

On the software front, a robust backend environment was developed using Node.js and Express.js, providing a scalable RESTful API for bidirectional communication between the hardware node and the centralized user dashboard. A MongoDB Atlas cluster was employed for data persistence, utilizing a document-oriented structure for the efficient retrieval of sensor logs, driver schedules, and user credentials. Data transmission occurs over a secured 2.4 GHz Wi-Fi network, with the ESP32 transmitting encrypted payloads. Furthermore, all API requests are authenticated using JSON Web Tokens (JWT) to mitigate unauthorized access attempts.

During the experimental phase, the system was calibrated for high-frequency sensor polling, incorporating algorithmic filtering to discard anomalous data spikes caused by uneven waste surfaces. The sanitized data is transmitted to a frontend web application developed with HTML5, CSS3, and JavaScript, and hosted on the Vercel platform. The interface dynamically updates the real-time status of the bins without requiring manual page refreshes, effectively functioning as a digital twin of the physical infrastructure.

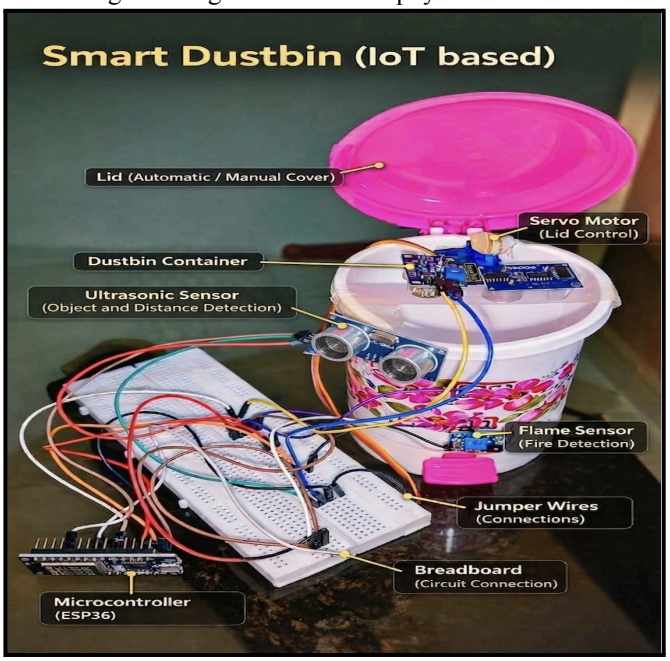

Fig 1. Physical Hardware Setup of the Smart Waste Bin

D. PERFORMANCE EVALUATION AND SYSTEM TESTING

To address real-world applicability and validate the operational efficacy of the proposed system, comprehensive empirical testing was conducted. The precision of the ultrasonic sensor was evaluated under varying waste accumulation levels, demonstrating a high measurement accuracy with an error margin of less than ±2 cm. Additionally, the data transmission latency between the ESP32 module and the MongoDB database was assessed. Under standard network conditions, the average latency was recorded between 150 and 200 milliseconds, ensuring near real-time synchronization. System robustness was further verified by simulating temporary network outages; the ESP32 successfully re-established connectivity and synchronized pending data without packet loss, proving the system's readiness for real-world deployment.

VI. METHODOLOGY

We built the system by connecting hardware components with cloud software. Our development process followed these specific steps:

A. Setting up the Hardware First, we worked on the data collection part. We used the ESP32 microcontroller because it has built-in Wi-Fi. We connected it to an HC-SR04 Ultrasonic Sensor. This sensor sends out sound waves to measure the distance to the garbage surface. We wrote code to convert this distance into a percentage (0% to 100% full). To prevent false readings due to uneven garbage, we added a simple filter in the code to smooth out the values. We also attached a Flame Sensor to the ESP32 to trigger an alert if it detects fire inside the bin.

B. Backend and Database Once the hardware was ready, we needed a place to send the data. We chose Node.js with Express.js for the server because it handles multiple

requests quickly. For storage, we used MongoDB Atlas. We designed a database schema that stores the Bin ID, Fill Level, Fire Status, and a Timestamp.

C. Data Transmission The ESP32 sends data to our server using HTTP POST requests. We created a specific API endpoint (/api/sensors/add) to receive this. To ensure security, the hardware sends a secure token (JWT) with every request. The server checks this token before saving any data.

D. Frontend Dashboard Finally, we created the user interface using HTML, CSS, and JavaScript. We hosted the frontend on Vercel. The dashboard automatically fetches fresh data from the API every few seconds. This allows the municipal authority to see the bin status change in real-time without refreshing the page.

A. Data Collection and Logging

The Data Collection and Logging phase is the backbone of the system, ensuring the accurate capture and persistence of environmental telemetry. At the hardware level, data is collected via an ESP32 microcontroller which polls the HCSR04 Ultrasonic sensor and Flame sensor at regular intervals. The raw distance values are converted into fill-level percentages, while the flame sensor monitors for binary hazard triggers. This data is then encapsulated into JSON payloads and transmitted through HTTP POST requests to the backend's /api/sensors/add endpoint.

On the server side, the Node.js/Express.js application validates the incoming data using JWT authentication to ensure that only authorized hardware nodes can log information. Once validated, each entry is timestamped and logged into a MongoDB Atlas collection. The database schema is designed for high-performance retrieval, storing critical fields such as binID, fillLevel, flameStatus, and timestamp. This systematic logging approach not only enables real-time monitoring on the frontend dashboard but also builds a historical dataset for long-term waste generation trend analysis and predictive scheduling.

B. Data Processing

The IoT Waste Management System operates through a seamless, end-to-end integration of hardware sensing, cloud computing, and real-time web visualization. The process initiates at the Physical Layer, where the ESP32 microcontroller acts as an edge gateway, continuously polling data from the HC-SR04 Ultrasonic sensor to measure waste volume and the Flame sensor to detect fire hazards. This raw telemetry is processed at the edge to filter signal noise and is then transmitted via a secured Wi-Fi connection as JSON payloads to the backend.

The Software Layer, powered by Node.js and Express.js, serves as the central intelligence hub. Upon receiving the data, the server validates the hardware's identity using JWT (JSON Web Tokens) and commits the records to a MongoDB Atlas cloud database. This ensures that the data is not only stored securely but is also accessible globally. Simultaneously, the Frontend Layer, built with HTML5, CSS3, and JavaScript and hosted on Vercel, fetches this live data through RESTful APIs. This allows Municipal Authorities and Drivers to monitor bin levels in real-time, receive instant emergency alerts, and optimize collection routes based on actual demand.

The entire ecosystem is unified by a CI/CD pipeline via GitHub, ensuring that any software updates are automatically deployed to the live environment. By bridging the gap between physical sensors and a cloud-hosted dashboard, the system automates the waste lifecycle—from detection and logging to visualization and proactive collection—making urban sanitation more efficient and data-driven.

.

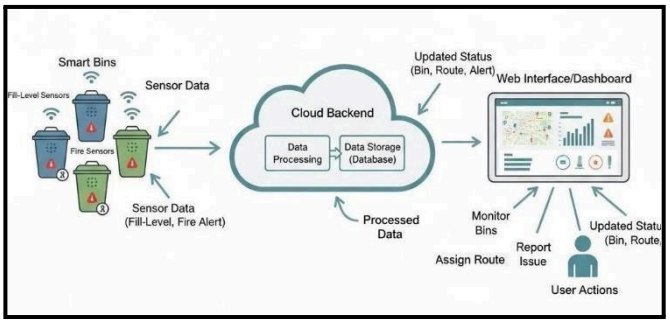

Fig 2. Workflow diagram

C. User Authentication and Role-Based Access Control

Security is a key part of our system. As shown in the login interface figure, we designed a simple but secure entry point for all users. The main feature here is the "Role Selection" dropdown menu. Before logging in, a user must choose their identity: Municipal Authority, Driver, or Citizen. This step is crucial because it tells the backend which permissions to grant.

Below the role selection, users enter their email and password. We also included a "Register" option for new users and a "Test Backend" button on the top bar to quickly check if the server is online. When a user clicks the "Login" button, the system sends their credentials to the Node.js server. If the email and password match, the server generates a unique JSON Web Token (JWT).

Once authenticated, the system automatically redirects the user to the correct dashboard:

- Authorities see the main analytics page with all bin data.
- Drivers see the route map.
- Citizens see the grievance reporting page.

This logic ensures that sensitive data remains safe and users only see what is relevant to them.

.

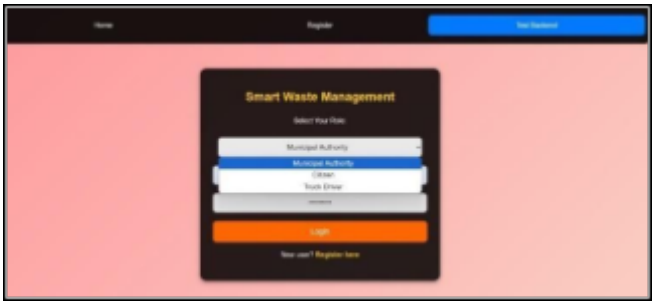

Fig 3. Unified Login Interface with Role-Based Access Selection

The images illustrate the Role-Based Authentication flow. Users first interact with a dynamic dropdown to select one of three designations: Municipal Authority, Citizen, or Truck Driver.

Once a role is selected, the interface captures the choice to trigger the Role-Based Access Control (RBAC) logic. This ensures that after a successful login, the Node.js backend redirects the user to their specific dashboard—such as waste analytics for authorities or route maps for drivers—ensuring a secure and personalized entry into the system.

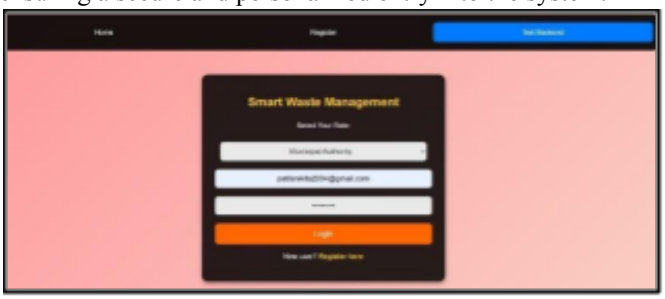

Fig 4. Interactive Role Selection Menu for Stakeholder Identification

After a successful login, the user is taken to the main Monitoring Dashboard. This page fetches the latest data directly from our MongoDB Atlas database and shows it in a simple table format.

As seen in the output image, the table displays the most important details:

- Bin Location: To know exactly where the garbage is.
- Fill Level (%): To check how much space is left in the bin.
- Flame Status: To see if there is any fire warning.

This direct connection between the hardware sensors and the website helps municipal officers make quick decisions. Instead of sending trucks blindly, they can now check this dashboard and send vehicles only to the locations that are actually full.

.

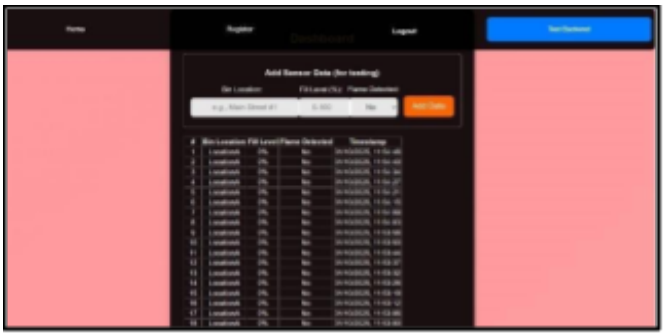

Fig 5. Real-Time Monitoring Dashboard with Live Sensor Telemetry and Timestamp Logs

VII. CONCLUSION AND FUTURE WORK

Conclusion We have successfully developed an IoT-based system that replaces the traditional, manual way of collecting garbage. By connecting ultrasonic sensors and flame sensors to the cloud, our project ensures that waste collection is done based on actual need, not just a fixed schedule. The testing phase confirmed that the ESP32 microcontroller accurately sends data to the MongoDB database using our Node.js backend. We also secured the platform using JWT, ensuring that authorities, drivers, and citizens have safe and separate access to the system. This project proves that technology can significantly reduce fuel wastage and operational costs while keeping the city safer from fire hazards.

Future Work While the current system works well as a prototype, we plan to add more features in the future to make it even better:

- GPS Integration: Currently, drivers have to check the dashboard. In the future, we want to link this with Google Maps to automatically guide trucks to the full bins.
- Automatic Segregation: We aim to add sensors that can separate wet and dry waste automatically at the source.
- Mobile Application: Right now, we have a website. Developing a dedicated mobile app would make it easier for drivers and citizens to use the system on the go.

VII. REFERENCES

[1] S. S. Navghane, M. S. Killedar, and V. M. Rohokale, "IoT Based Smart Garbage and Waste Collection Bin," *Int. J. Adv. Res. Electron. Commun. Eng.*, vol. 5, no. 5, 2016.

[2] S. Longhi, D. Marzioni, E. Alidori, G. DiDomenico, M. Palmerini, and L. Gambi, "Solid Waste Management Architecture Using Wireless Sensor Networks," *IEEE Sensors Journal*, 2012.

[3] T. Anagnostopoulos, A. Zaslavsky, K. Kolomvatsos, A. Medvedev, P. Amiriparian, J. Morley, and S. Hadjiefthymiades, "Challenges and Opportunities of Waste Management in Smart Cities," *IEEE Transactions on Sustainable Computing*, 2017.

[4] J. Gubbi, R. Buyya, S. Marusic, and M. Palaniswami, "Internet of Things (IoT): A Vision, Architectural Elements, and Future Directions," *Future Generation Computer Systems*, 2013.

[5] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications," *IEEE Communications Surveys & Tutorials*, 2015.

[6] A. Zanella, N. Bui, A. Castellani, L. Vangelista, and M. Zorzi, "Internet of Things for Smart Cities," *IEEE Internet of Things Journal*, 2014.

[7] S. M. R. Islam, D. Kwak, M. D. H. Kabir, M. Hossain, and K. S. Kwak, "The Internet of Things for Health Care: A Comprehensive Survey," *IEEE Access*, 2015.

[8] S. Kumar and P. Jain, "Smart Waste Management System Using IoT," *International Journal of Engineering Research & Technology (IJERT)*, 2018.

[9] M. Patel, J. Doshi, and J. Vala, "IoT Based Smart Garbage Monitoring System," *International Journal of Computer Applications*, 2017.

[10] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, University of California, Irvine, 2000.

[11] D. Hardt, "The OAuth 2.0 Authorization Framework," IETF, RFC 6749, 2012.

[12] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF, RFC 7519, 2015.

[13] K. Chodorow, *MongoDB: The Definitive Guide*, 2nd edition, O'Reilly Media, 2013.

[14] S. Tilkov and S. Vinoski, "Node.js: Using JavaScript to Build High-Performance Network Programs," *IEEE Internet Computing*, 2010.

[15] L. Atzori, A. Iera, and G. Morabito, "The Internet of Things: A Survey," *Computer Networks*, 2010.

[16] G. P. Borojeni and A. Abdi, "IoT-WMS: Design and Implementation of an IoT-based Smart Waste Management System," *2024 8th International Conference on Smart Cities, Internet of Things and Applications (SCIoT)*, 2024.

[17] S. R. Laha, B. K. Pattanayak, and S. Kumar, "A Smart Waste Management System Framework Using IoT and LoRa for Green City Project," *Int. J. Recent Innov. Trends Comput. Commun.*, 2023.

[18] M. Gurupriya and K. Yadav, "A Smart Waste Management System Using ESP8266," *2024 4th International Conference on Sustainable Expert Systems (ICSES)*, 2024.

[19] G. Uthayakumar et al., "Smart Waste Revolution: AI-Powered Biomedical Segregation and Recycling with IoT Integration," *SSRG Int. J. Mech. Eng*, 2024.

[20] A. S. Alowayr, "A Proposed IoT-Enabled Smart Waste Bin Management System and Efficient Route Selection," *IEEE Access*, 2023.